

ÉCOLE NATIONALE SUPÉRIEURE DES
TECHNOLOGIES
DE L'INFORMATION ET DE LA
COMMUNICATION

ENASTIC
Année scolaire 2025–2026

Prédiction du Décrochage
Scolaire Universitaire

À partir de données administratives et comportementales

Pipeline : Feature Engineering → Random Forest → Réseau de
Neurones → Calibration → Dashboard Streamlit

Auteure Haoua Chabadine Youssouf

Établissement ENASTIC

Filière Informatique

Année scolaire 2025–2026

Sujet n° 13

Table des matières

1	Introduction et contexte	2
1.1	Contexte général	2
1.2	Objectifs du projet	2
1.3	Variable cible	2
2	Description du dataset	3
2.1	Structure générale	3
2.2	Description des colonnes	3
2.3	Répartition de la variable cible	4
2.4	Réalisme des données	4
3	Feature Engineering	5
3.1	Pourquoi créer de nouvelles variables ?	5
3.2	Les 4 variables engineered	5
3.2.1	trend_notes — Tendance des notes	5
3.2.2	ratio_engagement — Score global d'implication	5
3.2.3	ratio_absence_connexion — Signal d'alarme	5
3.2.4	score_risque — Score composite	6
4	Modélisation	7
4.1	Préparation des données	7
4.2	Modèle 1 — Random Forest	7
4.2.1	Configuration	7
4.2.2	Principe	7
4.3	Modèle 2 — Réseau de neurones (MLP)	8
4.3.1	Architecture	8
4.3.2	Configuration	8
4.4	Calibration des probabilités	8
5	Résultats et évaluation	10
5.1	Comparaison des modèles	10
5.2	Rapport de classification — Random Forest	10
5.3	Importance des variables	11
6	Simulation intervention / groupe témoin	12
6.1	Protocole de la simulation	12

6.2	Modélisation de l'accompagnement	12
6.3	Résultats de la simulation	12
7	Tableau de bord — Streamlit	14
7.1	Choix technologique	14
7.2	Architecture du dashboard	14
7.2.1	Chargement des données	14
7.2.2	Les 5 sections	15
7.2.3	Fonctionnalités clés	15
7.2.4	Lancement	15
8	Conclusion	16
8.1	Ce que le modèle a appris	16
8.2	Limites du projet	16
8.3	Perspectives	16

Chapitre 1

Introduction et contexte

1.1 Contexte général

Le décrochage scolaire universitaire est un problème majeur dans les établissements d'enseignement supérieur africains. Un étudiant qui abandonne ses études représente non seulement un échec personnel, mais aussi un coût important pour l'institution et pour la société.

Ce projet a pour objectif de construire un système de prédiction du décrochage scolaire à partir de données administratives et comportementales, afin de permettre aux responsables de promotion d'identifier les étudiants en danger et d'intervenir avant qu'il ne soit trop tard.

1.2 Objectifs du projet

- Construire un dataset réaliste de 500 étudiants avec des profils variés
- Appliquer du feature engineering pour créer des variables plus informatives
- Entraîner et comparer deux modèles : Random Forest et Réseau de neurones (MLP)
- Calibrer les probabilités pour qu'elles soient fiables et interprétables
- Évaluer l'impact d'une intervention ciblée sur un groupe témoin
- Déployer un tableau de bord Streamlit pour le responsable de promotion

1.3 Variable cible

Chaque étudiant est classé dans l'une des trois catégories suivantes :

Catégorie	Signification
Diplômé	L'étudiant termine ses études avec succès
Décrochage	L'étudiant abandonne ses études
Inscrit	L'étudiant est encore en cours — situation incertaine

Chapitre 2

Description du dataset

2.1 Structure générale

Le fichier principal `student_dropout_500.csv` contient 500 étudiants décrits par 18 colonnes. Les colonnes `id`, `nom` et `prenom` servent uniquement à l'identification et sont retirées avant l'entraînement des modèles.

2.2 Description des colonnes

Variable	Type	Description
<code>age</code>	entier	Âge de l'étudiant (18 à 24 ans)
<code>sexe</code>	texte	M ou F
<code>filiere</code>	texte	info / telecom / mtic
<code>bourse</code>	texte	oui / non
<code>frais_payes</code>	texte	oui / non
<code>moyenne_sem1</code>	décimal	Moyenne semestre 1 (2.0 à 18.5)
<code>moyenne_sem2</code>	décimal	Moyenne semestre 2 (2.0 à 18.5)
<code>note_examen</code>	décimal	Note d'examen (2.0 à 18.5)
<code>absences</code>	entier	Nombre d'absences (0 à 20)
<code>retards</code>	entier	Nombre de retards (0 à 15)
<code>connexions_lms</code>	entier	Connexions à la plateforme (0 à 40)
<code>devoirs_rendus</code>	entier	Devoirs rendus (0 à 10)
<code>participation</code>	entier	Participation en classe (0 à 10)
<code>taux_presence</code>	entier	Taux de présence en % (10 à 99)
<code>abandon</code>	texte	Variable cible

2.3 Répartition de la variable cible

Statut	Nombre	Pourcentage
Diplômé	220	44.0%
Décrochage	152	30.4%
Inscrit	128	25.6%
Total	500	100%

2.4 Réalisme des données

Le dataset a été construit avec trois niveaux de réalisme pour éviter les données trop parfaites qui rendraient la prédiction triviale.

Trois profils d'étudiants ont été définis :

- **Profil stable (40%)** : bonnes notes (12–18), peu d'absences, engagement élevé sur le LMS
- **Profil moyen (30%)** : notes intermédiaires (9–12), zone grise — peut décrocher ou réussir selon les circonstances
- **Profil à risque (30%)** : faibles notes (4–9), absences élevées, peu d'engagement

Des cas contre-intuitifs ont été introduits intentionnellement :

- **Bons étudiants qui décrochent** : `frais_payes = non` malgré de bonnes notes
— raison financière
- **Mauvais étudiants qui persistent** : très engagés sur le LMS (`connexions_lms` élevé) malgré de faibles notes

Si toutes les données étaient parfaites, le modèle ne prédirait rien — il mémoriserait juste une règle évidente.

Chapitre 3

Feature Engineering

3.1 Pourquoi créer de nouvelles variables ?

Les variables brutes du dataset ne capturent pas tous les patterns utiles. Par exemple, un étudiant avec 10 absences et 30 connexions LMS est très différent d'un étudiant avec 10 absences et 2 connexions. Le ratio entre les deux capture cette différence mieux que les deux variables séparées.

Quatre nouvelles variables ont été construites :

3.2 Les 4 variables engineered

3.2.1 `trend_notes` — Tendence des notes

$$\text{trend_notes} = \text{moyenne_sem2} - \text{moyenne_sem1}$$

Si le résultat est positif, l'étudiant progresse entre les deux semestres. S'il est négatif, il régresse. Cette variable capture une dynamique que les deux moyennes prises séparément ne montrent pas.

3.2.2 `ratio_engagement` — Score global d'implication

$$\text{ratio_engagement} = \frac{\text{connexions_lms}}{40} \times 0,4 + \frac{\text{devoirs_rendus}}{10} \times 0,4 + \frac{\text{participation}}{10} \times 0,2$$

Ce score entre 0 et 1 résume l'implication globale de l'étudiant. On a choisi de pondérer les connexions LMS et les devoirs à 40% chacun, la participation à 20% car elle est plus subjective.

3.2.3 `ratio_absence_connexion` — Signal d'alarme

$$\text{ratio_absence_connexion} = \frac{\text{absences}}{\text{connexions_lms} + 1}$$

Plus ce ratio est élevé, plus l'étudiant est absent sans se connecter à la plateforme — c'est un signal fort de désengagement total. On ajoute 1 au dénominateur pour éviter

la division par zéro.

3.2.4 `score_risque` — Score composite

$$\text{score_risque} = \frac{(20 - m_1) \times 0,20 + (20 - m_2) \times 0,20 + \text{absences} \times 0,25 + (10 - \text{devoirs}) \times 0,15}{20}$$

Ce score synthétique entre 0 et 1 résume le niveau de risque global de l'étudiant en combinant six facteurs pondérés.

Ces 4 variables engineered se retrouvent parmi les **5 plus importantes** selon le Random Forest, ce qui valide l'intérêt de cette étape.

Chapitre 4

Modélisation

4.1 Préparation des données

Avant d'entraîner les modèles, les étapes suivantes sont appliquées :

1. Suppression des colonnes non prédictives : `id`, `nom`, `prenom`
2. Encodage des variables catégorielles avec `LabelEncoder` (sexe, filière, bourse, frais_payes)
3. Séparation train/test : **75% entraînement** (375 lignes) / **25% test** (125 lignes)
4. Stratification pour conserver la même répartition des classes dans les deux parties
5. Normalisation avec `StandardScaler` appliquée uniquement pour le réseau de neurones

4.2 Modèle 1 — Random Forest

4.2.1 Configuration

Listing 4.1 – Paramètres du Random Forest

```
1 rf_base = RandomForestClassifier(  
2     n_estimators=200,      # 200 arbres pour un resultat stable  
3     max_depth=8,          # limite la profondeur pour eviter  
4     min_samples_split=5,  le surapprentissage  
5     min_samples_leaf=2,  
6     class_weight="balanced", # compense le desequilibre entre  
7     random_state=42        classes  
8 )
```

4.2.2 Principe

Le Random Forest construit 200 arbres de décision indépendants, chacun entraîné sur un sous-ensemble aléatoire des données. La prédiction finale est le vote majoritaire de tous les arbres. Cela le rend robuste au surapprentissage et capable de gérer des variables de natures différentes.

4.3 Modèle 2 — Réseau de neurones (MLP)

4.3.1 Architecture

Couche	Neurones	Rôle
Entrée	18	Variables d'entrée
Cachée 1	128	Extraction des patterns principaux
Cachée 2	64	Réduction progressive
Cachée 3	32	Représentation finale
Sortie	3	Diplômé / Décrochage / Inscrit

4.3.2 Configuration

Listing 4.2 – Paramètres du réseau de neurones

```

1 mlp_base = MLPClassifier(
2     hidden_layer_sizes=(128, 64, 32),
3     activation="relu",
4     solver="adam",
5     alpha=0.001,           # regularisation L2
6     learning_rate="adaptive",
7     max_iter=500,
8     early_stopping=True,   # arrete si plus d'amelioration
9     validation_fraction=0.15,
10    random_state=42
11 )

```

4.4 Calibration des probabilités

La calibration est une étape souvent négligée mais essentielle. Sans elle, une probabilité de 70% prédite par le modèle ne correspond pas nécessairement à 70% de risque réel.

On teste deux méthodes avec `CalibratedClassifierCV` :

- **Isotonic** : méthode non-paramétrique, plus flexible
- **Sigmoid (Platt)** : méthode paramétrique, plus stable sur de petits échantillons

On mesure la qualité des probabilités avec le **Brier Score** (0 = parfait, 0.25 = aléatoire). La méthode avec le Brier Score le plus bas est retenue automatiquement.

$$\text{Brier Score} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Chapitre 5

Résultats et évaluation

5.1 Comparaison des modèles

Modèle	Accuracy	AUC	F1-CV	Brier	Log-Loss
Random Forest (brut)	75.2%	0.862	0.732	0.0876	0.668
RF + Calibration Isotonic	—	—	—	0.0842	—
Réseau de neurones (brut)	72.8%	0.859	0.577	0.0919	0.674
MLP + Calibration Sigmoid	—	—	—	0.0873	—

Le **Random Forest** est retenu comme modèle principal avec un AUC de **0.862**.
La calibration Isotonic améliore le Brier Score de 0.0876 à **0.0842**.

5.2 Rapport de classification — Random Forest

Classe	Précision	Rappel	F1-score	Support
Diplômé	0.80	0.82	0.81	55
Décrochage	0.86	0.82	0.84	38
Inscrit	0.55	0.56	0.55	32
Moyenne pondérée	0.75	0.75	0.75	125

La classe *Inscrit* obtient un F1 plus faible (0.55) car c'est la catégorie la plus ambiguë — un étudiant encore inscrit peut basculer dans l'un ou l'autre sens. C'est un comportement réaliste et attendu.

5.3 Importance des variables

Rang	Variable	Importance	Type
1	score_risque	19.8%	Construite (<i>FE</i>)
2	absences	18.2%	Originale
3	ratio_absence_connexion	16.8%	Construite (<i>FE</i>)
4	connexions_lms	15.5%	Originale
5	ratio_engagement	12.9%	Construite (<i>FE</i>)
6	taux_presence	10.9%	Originale
7	moyenne_seml	9.5%	Originale

3 des 5 premières variables sont issues du feature engineering (*FE*), ce qui confirme la pertinence de cette étape dans le pipeline.

Chapitre 6

Simulation intervention / groupe témoin

6.1 Protocole de la simulation

Pour évaluer l'impact d'une intervention ciblée, les 127 étudiants en zone rouge (probabilité $\geq 65\%$) ont été divisés en deux groupes tirés au sort :

- **Groupe intervention (63 étudiants)** : accompagnement simulé
- **Groupe témoin (64 étudiants)** : aucun changement

6.2 Modélisation de l'accompagnement

L'accompagnement simulé se traduit par les modifications suivantes :

Variable modifiée	Changement	Justification
absences	-4	Tuteur qui suit les présences
connexions_lms	+8	Accès renforcé à la plateforme
devoirs_rendus	+2	Suivi et relance des devoirs
participation	+2	Implication en classe renforcée

Après modification, les 4 variables engineered (`trend_notes`, `ratio_engagement`, `ratio_absence_connexion`, `score_risque`) sont recalculées avant de repasser dans le modèle.

6.3 Résultats de la simulation

Indicateur	Résultat
Étudiants à risque élevé total	127
Groupe intervention	63 étudiants
Groupe témoin	64 étudiants
P(décrochage) moyen avant intervention	$\approx 82\%$
P(décrochage) moyen après intervention	$\approx 63.6\%$
Réduction du risque moyen	-18.4%
Étudiants sortis de la zone rouge	21 / 63 (33%)
Évolution groupe témoin	0% (aucun changement)

Une intervention ciblée permettrait de faire passer environ **un tiers** des étudiants en danger sous le seuil critique de 65%. Le groupe témoin ne montre aucune amélioration, ce qui confirme la valeur ajoutée de l'accompagnement.

Chapitre 7

Tableau de bord — Streamlit

7.1 Choix technologique

Streamlit a été choisi pour développer le tableau de bord car :

- C'est un outil Python natif, **cohérent avec l'ensemble du pipeline ML**
- Il permet de créer une interface interactive sans connaître le développement web
- Il est utilisé en entreprise pour présenter des modèles ML
- Le code reste lisible et maintenable

7.2 Architecture du dashboard

7.2.1 Chargement des données

Le dashboard charge deux fichiers CSV complémentaires et les fusionne :

Listing 7.1 – Chargement et fusion des données

```
1 # source principale : les 500 étudiants
2 dropout = pd.read_csv("student_dropout_500.csv")
3
4 # probabilités calculées par le modèle ML
5 at_risk = pd.read_csv("students_at_risk.csv")
6
7 # on fusionne via la colonne id pour avoir les 500 avec leurs
   niveaux de risque
8 dropout = dropout.merge(at_risk[cols_risque], on="id", how="
   left")
```

Problème identifié et corrigé : si on chargeait uniquement `students_at_risk.csv` (127 lignes), les filtres « zone orange » et « zone verte » retournaient 0 résultats. La fusion sur le dataset complet résout ce problème.

7.2.2 Les 5 sections

Section	Contenu
Vue générale	4 KPI (total, zone rouge, orange, verte) + graphique statuts + camembert niveaux
Étudiants à risque	Tableau des 500 avec filtres par niveau / filière / nom
Performance modèle	Métriques RF et MLP + rapport de classification
Simulation intervention	KPI + graphique avant/après pour le groupe traité
Variables importantes	Barres d'importance colorées (rouge = risque, vert = protection)

7.2.3 Fonctionnalités clés

- **Sidebar de navigation** avec `st.radio` — change de section sans recharger la page
- **Cache des données** avec `@st.cache_data` — les CSV ne sont lus qu'une seule fois
- **Filtres interactifs** — par niveau de risque, filière, et recherche par nom ou prénom
- **Barre de progression** avec `st.column_config.ProgressColumn` — la probabilité de décrochage s'affiche visuellement
- **Gestion d'erreur** — si un CSV est absent, un message clair indique lequel manque

7.2.4 Lancement

Pour démarrer le dashboard, on exécute dans le terminal depuis le dossier du projet :

```
streamlit run dashboard_streamlit.py
```

Le dashboard s'ouvre automatiquement dans le navigateur à l'adresse `http://localhost:8501`.

Chapitre 8

Conclusion

8.1 Ce que le modèle a appris

- Les **absences** et le **ratio absence/connexion** sont les signaux d'alarme les plus forts
- L'**engagement sur la plateforme LMS** est un facteur protecteur important — parfois plus déterminant que les notes
- Les **variables construites par feature engineering** sont parmi les plus informatives (3 des 5 premières)
- La classe *Inscrit* est la plus difficile à prédire ($F1 = 0.55$), ce qui est attendu et réaliste

8.2 Limites du projet

- Le dataset est synthétique — les résultats réels pourraient différer avec de vraies données universitaires
- La simulation d'intervention est simplifiée et ne tient pas compte de facteurs externes (motivation personnelle, contexte familial)
- La classe *Inscrit* nécessite des améliorations ($F1 = 0.55$)

8.3 Perspectives

- Collecter de vraies données académiques pour valider le modèle
- Tester d'autres algorithmes : XGBoost, SVM, ou modèles d'ensemble
- Intégrer une explication individuelle par étudiant avec SHAP values
- Déployer le dashboard sur un serveur accessible à l'administration